Now you're ready to run the following SQL from psql. Note that the SCRUB_EMAIL() is now surrounding the users.email field. Type \df scrub* to make sure your scrubbed functions are listed.

sql/scrubbing_on_the_fly.sql

```
INSERT INTO users_copy(
  id, first_name, last_name,
  email, type, created_at, updated_at
)
(
  SELECT
  id, first_name, last_name,
  scrub_email(email), -- scrubber function
  type, created_at, updated_at
  FROM users
);
```

Now try comparing the email data stored for the same user in the users table and the users_copy table.

```
SELECT email FROM users WHERE id = 1;
             email
--------------------------------
 Beau-Marvin-driver-0@email.com

SELECT email FROM users_copy WHERE id = 1;
             email
--------------------------------
 16acaa1aa55e432d69ea@email.com
```

The users_copy table email should be the scrubbed version!

Note that the SQL statement above has an explicit list of columns for the users table from Rideshare at the time of publication. Review the latest schema and make any necessary column additions or modifications to the statement by checking out the latest version of the project.

Inserting 20,000 rows with a value transformed on the fly above should have been pretty quick. You can actually speed things up further though. Speeding this process up is important when you're working with millions of rows of data.

Speeding Up Inserts For Table Copying

By using database functions inside PostgreSQL to perform scrubbing as opposed to scrubbing in client application Ruby code, you're removing the latency that's added when the client application communicates with the PostgreSQL server.